

Pseudocódigos practica 1

EDA

Simón Foguel Giraldo – Pascual Rincon Cardona

1. Pseudos de los TADs:

1.1 TAD PILA:

`push(x)` inserta x en el extremo activo
`pop()` elimina el elemento del extremo activo
 precondición: la pila no está vacía
`top()` devuelve el elemento del extremo activo sin eliminarlo
 precondición: la pila no está vacía
`empty()` indica si no hay elementos
`size()` cantidad de elementos
`clear()` elimina todos los elementos

1.2 TAD COLA:

`enqueue(x)` inserta x por el final
`dequeue()` elimina el elemento del frente
 precondición: la cola no está vacía
`front()` devuelve el elemento del frente sin eliminarlo
`back()` devuelve el último elemento insertado
`empty()` indica si no hay elementos
`full()` indica si se alcanzó la capacidad (solo capacidad fija)
`size()` cantidad de elementos

2. Pseudos de las representaciones bases:

2.1 Arreglo dinámico:

Algoritmo `push_back(obj)`
 si `theSize = theCapacity` entonces
 `resize()`
 `array[theSize] <- obj`
 `theSize <- theSize + 1`

Algoritmo `resize()`

 Efecto: duplica la capacidad reservada trasladando los elementos

`nuevaCapacidad <- si theCapacity = 0 entonces 16 si no theCapacity × 2`
 `nuevoArray <- reservar nuevaCapacidad posiciones`

```

para i desde 0 hasta theSize - 1:
    nuevoArray[i] <- array[i]      // traslado, no copia
theCapacity <- nuevaCapacidad
liberar array
array <- nuevoArray

```

```

Algoritmo pop_back()
    si theSize = 0 entonces
        lanzar Error("pop_back sobre un vector vacio")
    theSize <- theSize - 1

```

2.2 Lista enlazada:

Nodo:

```

    dato : Object
    next : referencia al siguiente nodo (nula si es el último)

```

```

Algoritmo push_front(obj)
    nuevo <- Nodo(obj, head.next)
    head.next <- nuevo
    si theSize = 0 entonces tail <- nuevo
    theSize <- theSize + 1

```

```

Algoritmo push_back(obj)
    si theSize = 0 entonces
        push_front(obj)
    terminar
    nuevo <- Nodo(obj, nulo)
    tail.next <- nuevo
    tail <- nuevo
    theSize <- theSize + 1

```

```

Algoritmo pop_front()
    si theSize = 0 entonces
        lanzar Error("pop_front sobre una lista vacia")
    viejo <- head.next
    head.next <- viejo.next
    si theSize = 1 entonces tail <- nulo
    liberar viejo
    theSize <- theSize - 1

```

3. Pseudos de las estructuras evaluadas:

3.1 Pila sobre arreglo dinámico:

```

push(x)  -> vec.push_back(x)
pop()    -> vec.pop_back()

```

top() -> vec.back()
empty() -> vec.empty()
size() -> vec.size()
clear() -> mientras no empty(): pop()

3.2 Pila sobre lista enlazada:

push(x) -> lista.push_front(x)
pop() -> lista.pop_front()
top() -> lista.front()
empty() -> lista.empty()
size() -> lista.size()
clear() -> mientras no empty(): pop()

3.3 Cola circular sobre arreglo de capacidad fija:

Algoritmo move(indice) // avance circular
indice <- (indice + 1) mod capacity

Algoritmo enqueue(x)
si theSize = capacity entonces
lanzar Error("enqueue sobre una cola llena")
move(end)
array[end] <- x
theSize <- theSize + 1

Algoritmo dequeue()
si theSize = 0 entonces
lanzar Error("dequeue sobre una cola vacia")
move(begin)
theSize <- theSize - 1

front() -> array[begin] precondición: theSize > 0
back() -> array[end] precondición: theSize > 0
empty() -> theSize = 0
full() -> theSize = capacity

3.4 Cola sobre lista enlazada:

enqueue(x) -> lista.push_back(x)
dequeue() -> lista.pop_front()
front() -> lista.front()
back() -> lista.back()
empty() -> lista.empty()
size() -> lista.size()

4. Pseudo del problema 1:

4.1 TAD documento:

Algoritmo Modificar(pos, cuantosQuitar, textoNuevo)

Efecto: elimina cuantosQuitar caracteres a partir de pos e inserta textoNuevo en ese mismo lugar

Precondición: $\text{pos} \leq |\text{texto}|$ y $\text{pos} + \text{cuantosQuitar} \leq |\text{texto}|$

```
si pos > |texto| entonces
  lanzar Error("la posicion no esta en el documento")
si pos + cuantosQuitar > |texto| entonces
  lanzar Error("la cantidad a eliminar excede el tamaño del documento")
texto <- texto[0 .. pos-1] + textoNuevo + texto[pos+cuantosQuitar .. |texto|-1]
```

Algoritmo Leer(pos, largo) -> secuencia de caracteres

Precondición: $\text{pos} \leq |\text{texto}|$ y $\text{pos} + \text{largo} \leq |\text{texto}|$

Efecto: ninguno

```
si pos > |texto| entonces
  lanzar Error("la posicion no esta en el documento")
si pos + largo > |texto| entonces
  lanzar Error("la cantidad a leer excede el tamaño del documento")
devolver texto[pos .. pos+largo-1]
```

Algoritmo ObtenerTexto() -> secuencia de caracteres

devolver texto // por referencia constante: sin copia

4.2 Registro de operación:

Tipo enumerado TipoOp = { INSERT, DELETE, REPLACE }

Registro:

tipo	: TipoOp	// etiqueta descriptiva, para el log
pos	: entero	// posición donde ocurrió la operación
antes	: cadena	// fragmento que la operación destruyó
despues	: cadena	// fragmento que dejó en su lugar

antes = texto[pos .. pos+k-1] inmediatamente ANTES de aplicarla
despues = texto[pos .. pos+m-1] inmediatamente DESPUÉS de aplicarla

4.3 Deshacer y rehacer:

Algoritmo Deshacer(doc, reg)

Efecto: doc queda idéntico a su estado previo a la operación de reg

doc.Modificar(reg.pos, |reg.despues|, reg.antes)

Algoritmo Rehacer(doc, reg)

Efecto: doc queda idéntico a su estado posterior a la operación de reg

doc.Modificar(reg.pos, |reg.antes|, reg.despues)

4.4 Analisis lexico:

Tipo enumerado TipoEvento = { EDICION, UNDO, REDO }

Evento:

```
clase    : TipoEvento
op       : TipoOp      // solo significativo si clase = EDICION
pos      : entero
cuantos  : entero      // caracteres a quitar; 0 en INSERT
contenido : cadena      // texto a insertar; Ø en DELETE
```

Algoritmo LeerEvento(linea) -> Evento

```
abrir un flujo de lectura sobre linea
si no se puede leer una primera palabra entonces
lanzar Error("linea vacia")
comando <- primera palabra
si comando = "UNDO" entonces devolver Evento{clase = UNDO}
si comando = "REDO" entonces devolver Evento{clase = REDO}
si comando != "EDIT" entonces
lanzar Error("comando desconocido: " + comando)
si no se puede leer una palabra entonces
lanzar Error("falta el tipo de edicion")
tipo <- esa palabra
si no se puede leer un entero entonces
lanzar Error("posicion invalida o ausente")
pos <- ese entero
según tipo:
"INSERT":
contenido <- resto de la línea, sin el espacio separador inicial
devolver Evento{EDICION, INSERT, pos, 0, contenido}
DELETE":
si no se puede leer un entero entonces
lanzar Error("falta la cantidad a borrar")
cuantos <- ese entero
devolver Evento{EDICION, DELETE, pos, cuantos, Ø}
"REPLACE":
si no se puede leer un entero entonces
lanzar Error("falta la cantidad a reemplazar")
cuantos <- ese entero
contenido <- resto de la línea, sin el espacio separador inicial
devolver Evento{EDICION, REPLACE, pos, cuantos, contenido}
otro:
lanzar Error("tipo de edicion desconocido: " + tipo)
```

Algoritmo LeerArchivo(ruta) -> (eventos, errores)

```
abrir el archivo; si no se puede, lanzar Error
eventos <- colección vacía ; errores <- colección vacía ; numero <- 0
mientras queden líneas:
  leer la siguiente línea ; numero <- numero + 1
  si la línea está vacía o empieza por '#' entonces continuar
  intentar:
    eventos.agregar(LeerEvento(línea))
  si falla con Error e:
    errores.agregar("línea " + numero + ": " + mensaje de e)
devolver (eventos, errores)
```

4.5 Motor:

Motor(Pila):

```
doc      : Documento
undo     : Pila de Registro
redo     : Pila de Registro
log      : colección de cadenas
efectivos : entero, inicialmente 0
noOps    : entero, inicialmente 0
```

Algoritmo AplicarEdicion(evento)

```
si evento.cuantos = 0 y evento.contenido = ∅ entonces
  log.agregar("edicion nula, se ignora")
terminar
antes <- doc.Leer(evento.pos, evento.cuantos)
doc.Modificar(evento.pos, evento.cuantos, evento.contenido)
undo.push(Registro{evento.op, evento.pos, antes, evento.contenido})
redo.clear()
```

Algoritmo HacerUndo()

```
si undo está vacía entonces
  log.agregar("Pila vacia, UNDO sin efecto")
noOps <- noOps + 1
terminar
reg <- undo.top()
undo.pop()
Deshacer(doc, reg)
redo.push(reg)
log.agregar("UNDO aplicado")
efectivos <- efectivos + 1
```

Algoritmo HacerRedo()

```
si redo está vacía entonces
```

```

log.agregar("Pila vacia, REDO sin efecto")
noOps <- noOps + 1
terminar
reg <- redo.top()
redo.pop()
Rehacer(doc, reg)
undo.push(reg)
log.agregar("REDO aplicado")
efectivos <- efectivos + 1

```

```

Algoritmo Procesar(eventos)
para i desde 0 hasta |eventos| - 1:
si eventos[i].clase = EDICION entonces
intentar:
AplicarEdicion(eventos[i])
si falla con Error e:
log.agregar("evento " + (i+1) + ": " + mensaje de e)
si no, si eventos[i].clase = UNDO entonces
HacerUndo()
si no:
HacerRedo()

```

4.6 Reporte:

```

Algoritmo Reportar(motor, erroresLectura, salida)
escribir en salida el estado final del documento
escribir los errores de lectura, o "ninguno" si no hubo
escribir el log de operaciones, línea por línea
escribir el resumen:
elementos en la pila Undo
elementos en la pila Redo
operaciones de historial efectivas
operaciones de historial sin efecto (no-op)

```

```

Algoritmo principal(argumentos)
si no se recibió la ruta del archivo entonces
informar el modo de uso por el canal de error
terminar con código 1
intentar:
(eventos, errores) <- LeerArchivo(ruta)
motor <- Motor con pila sobre arreglo
motor.Procesar(eventos)
Reportar(motor, errores, salida estándar)
si falla con Error e:
informar el mensaje por el canal de error
terminar con código 1
terminar con código 0

```

5. Pseudo del problema 2:

5.1 Purga de marcas expiradas:

Algoritmo purgarExpiradas(t , marcas) $\rightarrow$ booleano

Entrada: t instante de llegada, marcas cola de marcas de tiempo

Efecto: elimina del frente toda marca anterior o igual a $t - T$

Salida: verdadero si el paquete cabe dentro del límite de tasa

limite $\leftarrow t - T$

mientras marcas no esté vacía y marcas.front() $\leq$ limite:

marcas.dequeue()

devolver |marcas| $< L$

5.2 Procesamiento de flujos:

Algoritmo procesarFlujo(paquetes, C , T , L , R)

buffer $\leftarrow$ ColaCircular de capacidad C // guarda instantes de salida

marcas $\leftarrow$ ColaEnlazada vacía // guarda instantes de llegada

aceptados $\leftarrow 0$; rechazadosLleno $\leftarrow 0$; rechazadosTasa $\leftarrow 0$

ultimoMuestreo $\leftarrow$ -INTERVALO_MUESTREO

arrancar cronómetro

para cada paquete (t , bytes) en paquetes:

// liberar el búfer de los paquetes ya servidos

mientras buffer no esté vacío y buffer.front() $\leq t$:

buffer.dequeue()

si buffer no está lleno y purgarExpiradas(t , marcas):

salidaAnterior $\leftarrow$ si buffer está vacío entonces t si no buffer.back()

buffer.enqueue(max(t , salidaAnterior) + bytes / R)

marcas.enqueue(t)

aceptados $\leftarrow$ aceptados + 1

si no, si buffer está lleno:

rechazadosLleno $\leftarrow$ rechazadosLleno + 1

si no:

rechazadosTasa $\leftarrow$ rechazadosTasa + 1

si $t - \text{ultimoMuestreo} \geq \text{INTERVALO_MUESTREO}$:

registrar muestra (t , aceptados, rechazadosLleno, rechazadosTasa, |buffer|)

ultimoMuestreo $\leftarrow t$

detener cronómetro

reportar totales y volcar las muestras a archivo

5.3 Orquestación:

Algoritmo principal()

leer el archivo de paquetes a memoria

abrir el archivo de parámetros

si no se pudo abrir, terminar con código de error

para cada línea (C , T , L , R) del archivo de parámetros:

procesarFlujo con la cola circular

procesarFlujo con la cola enlazada